# Utilisation de Scapy en Python : Explications détaillées

Scapy est une bibliothèque Python très utilisée en cybersécurité pour manipuler les paquets réseau. Voici une explication approfondie de ses principales fonctionnalités :

# 1.Commandes de base dans Scapy

Voici un résumé des commandes essentielles de Scapy pour explorer les protocoles disponibles, leurs usages et la découverte des fonctions :

## 1.1. Découvrir les protocoles et couches disponibles

- **`ls()`** : Affiche les champs d'une couche ou d'un protocole.

  Exemple :

  ```
  ls(IP)      # Affiche les champs du protocole IP
  ls(TCP)     # Affiche les champs du protocole TCP
  ls(Ether)   # Affiche les champs de la couche Ethernet
  ```

  Utilise cette commande pour voir quels paramètres tu peux personnaliser lors de la création d'un paquet.

- **`lsc()`** : Liste toutes les commandes Scapy disponibles (fonctions principales).

  Exemple :

  ```
  lsc()
  ```

  Cela te donne un aperçu des fonctions utiles pour le sniffing, l'envoi, la lecture/écriture de fichiers pcap, etc.

- **`dir()`** : Affiche tous les attributs et méthodes d'un objet ou d'une classe.

  Exemple :

  ```
  dir(IP)
  dir(TCP)
  dir(paquet)
  ```

  Pratique pour explorer les méthodes et propriétés accessibles sur un paquet ou une couche.

## 1.2. Afficher la structure et les valeurs d'un paquet

- **`show()`** : Affiche la structure détaillée et les valeurs des champs d'un paquet.

  Exemple :

  ```
  paquet = IP(dst="8.8.8.8") / ICMP()
  paquet.show()
  ```

  Utile pour vérifier la composition d'un paquet avant de l'envoyer.

- **`summary()`** : Affiche un résumé compact d'un paquet ou d'une liste de paquets.

  Exemple :

  ```
  paquets = sniff(count=5)
  paquets.summary()
  ```

  Permet d'avoir une vue rapide des paquets capturés.

## 1.3. Utiliser l'aide intégrée

- **`help()`** : Affiche l'aide sur une fonction, une classe ou un protocole.

  Exemple :

  ```
  help(IP)
  help(sniff)
  help(ls)
  ```

  Pratique pour comprendre le rôle et les paramètres d'une fonction ou d'une couche.

## 1.4. Lister tous les protocoles/couches disponibles

- **`conf.l3types`** : Liste les couches de niveau 3 (IP, IPv6, etc.).
- **`conf.l2types`** : Liste les couches de niveau 2 (Ethernet, etc.).

  Exemple :

  ```
  print(conf.l3types)
  print(conf.l2types)
  ```

  Permet de voir tous les protocoles pris en charge par Scapy.

## 1.5 Résumé mnémotechnique

- **ls()** : Liste les champs d'un protocole
- **lsc()** : Liste les commandes Scapy
- **dir()** : Liste les attributs/méthodes d'un objet
- **show()** : Affiche la structure détaillée d'un paquet
- **summary()** : Résumé des paquets
- **help()** : Affiche l'aide sur une fonction ou un protocole
- **conf.l3types / conf.l2types** : Liste les couches disponibles

# 2. Création de paquets

Scapy permet de construire des paquets réseau en combinant différentes couches (Ethernet, IP, TCP, UDP, ICMP, etc.) grâce à l'opérateur `/`. Chaque couche possède des champs que l'on peut personnaliser.

**Exemple simple :**

```
from scapy.all import *
paquet = IP(dst="192.168.1.1") / ICMP()
paquet.show()  # Affiche tous les champs du paquet
```

- Ici, on crée un paquet IP dont la destination est `192.168.1.1`, puis on ajoute une couche ICMP (utilisée pour le ping).
- La méthode `show()` permet de visualiser la structure et les valeurs des champs du paquet.

**Personnalisation :**

```
paquet = IP(dst="192.168.1.1", ttl=128) / TCP(dport=80, flags="S")
```

- On modifie le champ `ttl` (Time To Live) et on crée un paquet TCP avec le drapeau SYN (pour initier une connexion).

**Création de plusieurs paquets :**

```
paquets = [IP(dst=f"192.168.1.{i}") / ICMP() for i in range(1, 10)]
```

- On génère une liste de paquets pour scanner plusieurs adresses IP.

# 3. Sniffing (capture de paquets)

Le sniffing consiste à écouter le trafic réseau en temps réel. Scapy permet de capturer les paquets qui transitent sur une interface réseau.

**Capture simple :**

```
paquets = sniff(iface="eth0", count=10)
paquets.summary()  # Affiche un résumé des paquets capturés
```

- On capture 10 paquets sur l'interface `eth0`.

**Filtrage par protocole :**

```
paquets = sniff(iface="eth0", filter="icmp", count=5)
```

- On ne capture que les paquets ICMP (utiles pour surveiller les pings).

**Traitement en temps réel :**

```
def analyse_paquet(pkt):
    pkt.show()
sniff(prn=analyse_paquet, count=3)
```

- À chaque paquet capturé, la fonction `analyse_paquet` est appelée pour afficher son contenu.

# 4. Envoi de paquets

Scapy permet d'envoyer des paquets sur le réseau, soit pour tester la réaction d'un système, soit pour simuler des attaques ou des scans.

**Envoi simple :**

```
send(IP(dst="192.168.1.1") / ICMP())
```

- Envoie un paquet ICMP (ping) à l'adresse spécifiée.

**Envoi et réception (ping avec réponse) :**

```
rep = sr1(IP(dst="192.168.1.1") / ICMP())
if rep:
    print("Réponse reçue :", rep.summary())
```

- `sr1()` envoie le paquet et attend une seule réponse. On peut ensuite analyser la réponse reçue.

**Envoi au niveau Ethernet :**

```
sendp(Ether() / IP(dst="192.168.1.1") / ICMP())
```

- `sendp()` permet d'envoyer des paquets au niveau de la couche 2 (Ethernet).

# 5. Exemples de scripts pour la cybersécurité

## a) Scan d'adresses IP (découverte d'hôtes)

Ce script envoie des paquets ICMP à une plage d'adresses pour détecter les machines actives.

```
from scapy.all import *
plage = "192.168.1.1-15"
paquet = Ether() / IP(dst=plage) / ICMP()
rep, non_rep = srp(paquet, timeout=0.5)
for r in rep:
  print(r[1][IP].src, "a répondu au ping")
```

- On utilise `srp()` pour envoyer des paquets et recevoir les réponses.
- Les adresses qui répondent sont considérées comme actives.

## b) Scan de ports TCP

Ce script teste l'ouverture de ports sur une machine cible.

```
from scapy.all import *
ip = "192.168.1.1"
for port in range(20, 25):
  pkt = IP(dst=ip) / TCP(dport=port, flags="S")
  rep = sr1(pkt, timeout=1, verbose=0)
  if rep and rep.haslayer(TCP) and rep[TCP].flags == 0x12:
    print(f"Port {port} ouvert")
```

- On envoie un paquet TCP SYN sur chaque port.
- Si la réponse a le drapeau SYN-ACK (`flags == 0x12`), le port est ouvert.

## c) Détection d'attaque ARP spoofing

Ce script surveille les paquets ARP pour détecter des réponses suspectes (souvent utilisées dans les attaques de type Man-in-the-Middle).

```
from scapy.all import *
def detect_arp(pkt):
  if pkt.haslayer(ARP) and pkt[ARP].op == 2:
    print(f"ARP reply: {pkt[ARP].psrc} is at {pkt[ARP].hwsrc}")
sniff(filter="arp", prn=detect_arp, store=0)
```

- On surveille les paquets ARP et affiche chaque réponse reçue.
- Cela permet de repérer des changements d'adresse MAC associés à une IP (signe d'une attaque).

# Résumé rapide

- **Création de paquets** : forge des paquets personnalisés pour tester ou simuler des protocoles.
- **Sniffing** : capture et analyse le trafic réseau en temps réel.
- **Envoi de paquets** : permet de tester la sécurité, simuler des attaques ou des scans.
- **Scripts cybersécurité** : automatisent la découverte d'hôtes, le scan de ports, la détection d'intrusions.